COMPANION BOOKLET · ALGORITHM ANALYSIS WITH PYTHON

Skiena Chapter 2 — Every Problem, Explained Simply

Every exercise from Chapter 2 (“Algorithm Analysis”) of Steven Skiena's *The Algorithm Design Manual*, restated in plain words and worked out step by step with tiny numbers. Nothing here assumes more than the Python taught in the first few weeks of the course. Where a textbook would write a proof, we tell the story instead.

~50 problems · no jargon left unexplained · print-friendly · answers in green

---

JUMP TO A SECTION

Program Analysis

Problem 2-1 PROGRAM ANALYSIS

IN PLAIN WORDS

Three loops are stacked inside each other, adding 1 to a counter `r`. Skiena asks two things: the exact value `r` ends up with (as a formula in `n`), and the Big-O running time.

```
def mystery(n):
    r = 0
    for i in range(1, n):              # i = 1 .. n-1
        for j in range(i + 1, n + 1):  # j = i+1 .. n
            for k in range(1, j + 1):  # k = 1 .. j
                r += 1
    return r
```

Count it exactly. The innermost loop `k = 1 .. j` runs `j` times, so it adds `j` to `r`. So really `r` is: for every allowed pair `(i, j)`, add `j`.

Turn it around and count by `j` instead. For a fixed `j` (which ranges from 2 to n), how many `i`'s reach it? The rule is `i < j`, so `i` can be `1, 2, …, j−1` — that is `j−1` values. Each of those contributes `j`. So the total is

$$r = 2\cdot 1 + 3\cdot 2 + 4\cdot 3 + \dots + n\cdot(n-1) = \sum_{j=2}^{n} j(j-1)$$

Adding those up gives a tidy closed form:

$$r = (n^3 - n) / 3$$

Check on a tiny case. For `n = 5`: (125 − 5)/3 = 120/3 = **40**, which is exactly what the function returns. For `n = 10`: (1000 − 10)/3 = 330. Both match.

The biggest piece of that formula is $n^3/3$, so the running time is **O(n³)**. (Doubling `n` multiplies the work by about 8 — the fingerprint of three nested loops that each grow with `n`.)

> ANSWER
>
> The function returns $r = (n^3 - n) / 3$, and the worst-case running time is **Θ(n³)**.

WHY IT MATTERS

The exercise asks for both the value *and* the growth — and the neat way to get the value is to re-count by the innermost variable instead of fighting the loop bounds.

Problem 2-2 PROGRAM ANALYSIS

IN PLAIN WORDS

Another three-loop function that counts up in `r`. This one looks trickier because the innermost loop's start and stop both move around. The question is the same: what does `r` come out to, and how slow is it?

Here it is in Python:

```
def pesky(n):
    r = 0
    for i in range(1, n + 1):              # i = 1 ... n
        for j in range(1, i + 1):          # j = 1 ... i
            for k in range(j, i + j + 1):  # k = j ... i+j
                r += 1
    return r
```

The scary-looking part is the inner loop: `k` goes from `j` up to `i + j`. So how many numbers is that? Count them: from `j` to `i + j` the count is `(i + j) - j + 1`. The two `j`s cancel out, leaving just `i + 1`.

So here is the nice surprise: the inner loop always runs `i + 1` times, no matter what `j` is. The wobbly start and stop were a distraction. The length never depended on `j` at all.

Now it is just three loops that all grow with `n`, so again the work piles up like `n` times `n` times `n`. That is cubic.

Let us check with tiny numbers. The tidy formula for this one turns out to be `n(n+1)(n+2)/3`. For `n = 5` that is 5 x 6 x 7 / 3 = 70. And if you run `pesky(5)`, you get exactly 70. For `n = 10` both give 440. For `n = 20` both give 3080. So the counting and the formula agree perfectly.

You do not need to memorise that formula. Just notice it multiplies three things that each grow like `n` (roughly `n` times `n` times `n`, divided by 3). The divide-by-3 is a fixed shrink that does not change the shape, so the shape is still cubic.

So the running time is **O(n³)**, which means ten times the input is about a thousand times the work.

> ANSWER

> The inner loop always runs `i + 1` times, so `pesky(n)` returns `n(n+1)(n+2)/3`, a cubic polynomial, and its running time is **O(n³)**.

WHY IT MATTERS

Messy-looking loop bounds often simplify to something clean once you count "how many numbers is that?" — do the subtraction before you panic.

Problem 2-3 PROGRAM ANALYSIS

IN PLAIN WORDS

Four loops nested inside each other. Give the exact value of `r` as a formula in `n`, and the Big-O time.

```
def prestiferous(n):
    r = 0
    for i in range(1, n + 1):
        for j in range(1, i + 1):
            for k in range(j, i + j + 1):
                for l in range(1, (i + j - k) + 1):
                    r += 1
    return r
```

Peel it from the inside out. The innermost loop `l = 1 .. (i+j-k)` runs `i+j-k` times. Now sum that over `k = j .. i+j`. As `k` climbs, `i+j-k` counts down: `i, i-1, …, 1, 0`. Adding `0 + 1 + … + i` gives `i(i+1)/2`.

Next, the `j = 1 .. i` loop repeats that `i(i+1)/2` a total of `i` times (it does not depend on `j`), giving `i²(i+1)/2`. Finally sum over `i = 1 .. n`:

$$r = \tfrac{1}{2} \sum_{i=1}^{n} (i^3 + i^2) = n(n+1)(n+2)(3n+1) \,/\, 24$$

Check: for `n = 1` the formula gives 1·2·3·4/24 = 1, and the function indeed returns 1. For `n = 2` it gives 2·3·4·7/24 = 7, which also matches.

The biggest piece is proportional to n^4, so the running time is **O(n⁴)**. (We did not just say "four loops, so n⁴" — that shortcut is not a proof, because loop bounds can change the answer. We actually summed the loops.)

> ANSWER
>
> `r = n(n+1)(n+2)(3n+1) / 24`, and the running time is **Θ(n⁴)**.

WHY IT MATTERS

Peeling the sums from the inside out gives the exact count; the loop *depth* only suggests the class, it does not prove it.

Problem 2-4 PROGRAM ANALYSIS

IN PLAIN WORDS

Three nested loops whose start points depend on the earlier counters. Give the exact value of `r` and the worst-case Big-O.

```
def conundrum(n):
    r = 0
    for i in range(1, n + 1):
        for j in range(i + 1, n + 1):
            for k in range(i + j - 1, n + 1):
                r += 1
    return r
```

Count the innermost loop. `k` runs from `i+j-1` to `n`, so it repeats `n - (i+j-1) + 1 = n - i - j + 2` times — but only when that number is positive (otherwise the loop does nothing). So only pairs with `i + j ≤ n + 1` contribute anything.

Doing the inner sum over `j` first, then over `i`, the surviving pairs give a clean count. Grouping by `i`:

$$r = \sum_{i=1}^{\lfloor n/2 \rfloor} (n - 2i + 1)(n - 2i + 2) / 2$$

which works out to (writing `m = ⌊n/2⌋`):

$$r = m(m+1)(4m-1)/6 \quad (n \text{ even}), \qquad m(m+1)(4m+5)/6 \quad (n \text{ odd})$$

Check: both forms agree with the brute-force count for every `n` up to 40. Only `i` up to about `n/2` contributes (beyond that the inner range is empty), which is why the constant is smaller than for a "full" triple loop.

The biggest piece grows like n^3, so the running time is **$\Theta(n^3)$**. Doubling `n` multiplies the work by about 8.

> ANSWER
>
> $r = \sum_{i=1}^{\lfloor n/2 \rfloor} (n-2i+1)(n-2i+2)/2$ (closed form above), and the worst-case running time is **$\Theta(n^3)$**.

WHY IT MATTERS

Loop *depth* is only a hint: here the moving start points empty out many iterations, so you must add up the ranges to be sure — the count, not the nesting, decides the class.

Problem 2-5 PROGRAM ANALYSIS

IN PLAIN WORDS

A polynomial is just a sum like `a₀ + a₁x + a₂x² + ...` — each term is a number times a power of `x`. To work out its value you have to do some multiplying and adding. The question has three parts: (a) how many multiplications and additions does the obvious method do in the worst case; (b) how many multiplications on average; (c) can we do it with fewer?

Here is the obvious method. It builds each power of `x` as it goes:

```
def evaluate_naive(a, x):
    # a is the list of numbers in front: a[0], a[1], ..., a[n]
    n = len(a) - 1
    p = a[0]
    xpower = 1
    for i in range(1, n + 1):
        xpower = x * xpower       # one multiplication
        p = p + a[i] * xpower     # one multiplication, one addition
    return p
```

(a) Worst case. Just count the arithmetic inside the loop. The line `xpower = x * xpower` is one multiply. The line `p = p + a[i] * xpower` is one multiply (`a[i] * xpower`) and one add. So every trip around the loop costs *two* multiplies and *one* add. The loop runs `n` times. So the totals are **2n multiplications and n additions**.

(b) Average case. Here is the quiet insight. Look at the loop: there is no `if`, no `break`, nothing that changes based on the actual numbers. The loop always runs the same `n` times and always does the same work each time. When the work never depends on the data, the average case cannot be any different from the worst case. So the average is also **2n multiplications**.

(c) Can we do better? Yes. The waste is that first multiply, the one that keeps rebuilding `xpower` from scratch. There is a smarter method called **Horner's rule**. Instead of building powers, it starts from the top number and folds one number in at a time:

```
def evaluate_horner(a, x):
    n = len(a) - 1
    p = a[n]                      # start from the top number
```

```
    for i in range(n - 1, -1, -1): # i = n-1, n-2, ..., 0
        p = p * x + a[i]           # one multiplication, one addition
    return p
```

Now the loop body is a single `p * x + a[i]`: **one multiply and one add per trip**, over `n` trips. That is **n multiplications and n additions**. The additions are the same as before, but the multiplications have been cut in half, from `2n` down to `n`.

Let us see both give the same answer on a tiny example. Take the numbers `[2, -3, 1, 4]` (which means the polynomial `2 - 3x + x² + 4x³`, so `n = 3`) and plug in `x = 2`. Both methods return 32. But the obvious method used 6 multiplies (that is 2n = 2 x 3), while Horner used only 3 (that is n = 3). Same answer, half the multiplying.

> ANSWER
>
> (a) The obvious method does **2n multiplications and n additions** in the worst case. (b) The average is also **2n multiplications**, because the loop never branches on the data, so average equals worst. (c) Yes — **Horner's rule** does it with only **n multiplications and n additions**, halving the multiplications.

WHY IT MATTERS

The same result can often be computed with far less work just by reorganising the arithmetic — Horner's rule is the standard, faster way every calculator and library evaluates polynomials.

Problem 2-6 PROGRAM ANALYSIS

IN PLAIN WORDS

Here is the usual way to find the biggest value in a list: remember the first item, then walk through the rest and keep whatever is bigger. You are asked to argue that this really does find the biggest, for *any* list — not just to trust it, but to explain why it can never go wrong.

The code:

```
m = A[0]                        # current champion
for i in range(1, len(A)):
    if A[i] > m:
        m = A[i]                # a bigger value takes the title
print(m)                        # claim: this is the biggest
```

Think of it as a sports tournament. `m` is the current champion — the biggest value we have seen so far. We start by crowning the first item as champion. Then each new item steps up as a challenger. If the challenger is bigger, it becomes the new champion. If not, the old champion keeps the belt.

To prove this always works, we lean on one simple sentence that stays true the whole way through. (A sentence that stays true through every loop trip has a name: a *loop invariant*.) Ours is:

After we have looked at the first k items, `m` holds the biggest of those k.

If that sentence is still true at the very end — when we have looked at every item — then `m` must hold the biggest of the whole list. So all we have to do is show it can never become false. Two little checks do it.

The start. Before the loop we set `m = A[0]`. With just one item looked at, the biggest of that one item is obviously itself. So the sentence is true at the start.

Each step keeps it true. Suppose the sentence is true after `k` items, so `m` is the biggest of the first `k`. Now the next item, `A[k]`, steps up. Only two things can happen:

- The newcomer is bigger — the `if` fires, `m` becomes `A[k]`, and since it beat the old champion (which had already beaten everyone before it), `m` is now the biggest of all `k + 1` items.
- The newcomer is not bigger — we leave `m` alone. It is still the biggest of the first `k`, and those were all at least as big as the newcomer, so `m` is still the biggest of all `k + 1`.

Either way the sentence survives the step. So it is true after one item, and each trip carries it safely from `k` items to `k + 1` items. Therefore it is still true after the last item — which means the `m` we print really is the biggest in the whole list.

Trace it once by hand on a tiny list, say `[3, 7, 2]`: champion starts at 3, then 7 beats it (champion becomes 7), then 2 loses (champion stays 7). Out comes 7, the biggest. That is exactly the running-champion argument, just done with real numbers.

> ANSWER
>
> The loop keeps the running-champion sentence "`m` is the biggest seen so far" true at every step: it is true after the first item, and each trip preserves it, so after the last item `m` is the biggest of the whole list. The algorithm is correct.

WHY IT MATTERS

The "one sentence that stays true every loop" idea is how programmers convince themselves (and each other) that a loop is right, instead of just hoping it is.

Big Oh Notation

First, the five words

All of Big-O is about one question: *as the input gets bigger, how fast does the work grow?* Not the exact number of steps — just the growth. Here are the five symbols in plain words. Whenever you see one, read the plain words instead.

SYMBOL	SAY IT AS	EVERYDAY MEANING
O	Big Oh	grows no faster than — a **ceiling**
Ω	Big Omega	grows no slower than — a **floor**
Θ	Big Theta	grows at the **same rate** (a ceiling and a floor at the same height)
o	little oh	**strictly slower** (below, and never catches up)
ω	little omega	**strictly faster** (above, and pulls away)

Two handy facts that come up again and again:

- **Θ = O and Ω together.** If something is both a ceiling and a floor, it pins the rate exactly.
- **Big-O ignores constant factors and smaller pieces.** `5n`, `n`, and `1000n` all count as the same: linear. And in `n² + n`, the little `+ n` does not matter — only the biggest piece, `n²`, counts.

The growth ladder (your everyday tool)
Almost every problem in this chapter is solved by this one picture. It lists common functions from *slowest-growing* to *fastest-growing*. The symbol `≪` means “grows slower than”.

$$1 \ll \log n \ll \sqrt{n} \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \ll 3^n \ll n!$$

The recipe for almost every true/false or relationship question: replace each side by where it sits on the ladder, then compare. If the left side is lower on the ladder, it is `O` of the right. If higher, it is `Ω`. If they land on the same rung, it is `Θ`.

A few ladder facts worth memorising, because they settle most questions instantly:

- Any **power of n** (even tiny `√n`) beats **any logarithm**.
- Any **exponential** (like `2ⁿ`) beats **any polynomial** (like `n³` or `n¹⁰⁰`).
- **n!** beats every fixed-base exponential.

- Bases matter for exponentials: $2^n \ll 3^n$. But adding a constant to n does *not* matter: $2^{n+1} = 2 \cdot 2^n$ is the same rate as 2^n.

Problem 2-7 BIG OH

IN PLAIN WORDS

Two yes/no questions. (a) Does 2^{n+1} (that is 2 to the power $n+1$) grow no faster than 2^n? (b) Does 2^{2n} (that is 2 to the power $2n$) grow no faster than 2^n?

(a) Rewrite it: $2^{n+1} = 2 \cdot 2^n$. That is just 2^n with a 2 stuck on the front — a constant factor. Big-O throws away constant factors, so this stays the same rate. At $n=10$: $2^{n+1} = 2048$ and $2^n = 1024$. Always exactly double, never runs away.

(b) Rewrite it: $2^{2n} = (2^n)^2 = 4^n$. That is a bigger base, 4 instead of 2. Their ratio is $4^n / 2^n = 2^n$, which itself shoots off to infinity. No fixed number c can hold 4^n below $c \cdot 2^n$. At $n=10$: $4^n = 1{,}048{,}576$ versus $2^n = 1024$ — already 1024 times bigger, and the gap keeps widening. Doubling the *exponent* changes the base; it is not a constant factor.

> ANSWER
>
> (a) True. (b) False.

WHY IT MATTERS

It teaches the single most common trap: a constant on the exponent ($+1$) is harmless, but multiplying the exponent ($2n$) secretly changes the base and breaks the bound.

Problem 2-8 BIG OH

IN PLAIN WORDS

For each pair of functions f and g, is f a ceiling (O), a floor (Ω), or exactly the same rate (Θ) as g? Just read each pair off the ladder. (When both O and Ω hold, the honest answer is Θ.)

	F	G	VERDICT	PLAIN REASON
(a)	$\log(n^2)$	$\log n + 5$	Θ	$\log(n^2) = 2 \log n$; both are just $\log n$ up to a constant — same rung
(b)	$\sqrt{n}$	$\log(n^2) = 2 \log n$	Ω	a power of n beats any log, so f is above g
(c)	$(\log n)^2$	$\log n$	Ω	squaring a growing thing makes it grow faster
(d)	n	$(\log n)^2$	Ω	a plain power of n beats any power of a log
(e)	$n \log n + n$	$\log n$	Ω	$n \log n$ sits far above $\log n$
(f)	10	$\log 10$	Θ	both are fixed numbers — neither grows, same rung (the bottom)
(g)	2^n	$10\, n^2$	Ω	an exponential beats any polynomial
(h)	2^n	3^n	O	2^n grows slower than 3^n (their ratio $(2/3)^n$ shrinks to 0)

> ANSWER
>
> (a) Θ · (b) Ω · (c) Ω · (d) Ω · (e) Ω · (f) Θ · (g) Ω · (h) O.

WHY IT MATTERS

It drills the ladder-reading skill you will lean on for every other problem in the chapter.

Problem 2-9 BIG OH

IN PLAIN WORDS

For each pair, which one is the ceiling of the other? Answer `f = O(g)` (f is the smaller/lower one), `g = O(f)` (g is the smaller one), or both (a tie, meaning Θ). Trick: simplify each function to its biggest piece, then compare on the ladder.

	F	G	VERDICT	PLAIN REASON
(a)	`(n² − n)/2`	`6n`	`g = O(f)`	`f` is basically `n²/2` (quadratic); `g` is only linear, so `g` is the smaller
(b)	`n + 2√n`	`n²`	`f = O(g)`	`f` is basically `n`, well below `n²`
(c)	`n log n`	`n√n / 2`	`f = O(g)`	`√n` grows faster than `log n`, so `n√n` beats `n log n`
(d)	`n + log n`	`√n`	`g = O(f)`	`f` is basically `n`, above `√n`
(e)	`2(log n)²`	`log n + 1`	`g = O(f)`	`(log n)²` grows faster than `log n`
(f)	`4n log n + n`	`(n² − n)/2`	`f = O(g)`	`n log n` sits below `n²`

None of these pairs lands on the same rung, so there is no Θ tie here.

> ANSWER
>
> (a) g=O(f) · (b) f=O(g) · (c) f=O(g) · (d) g=O(f) · (e) g=O(f) · (f) f=O(g).

WHY IT MATTERS

Simplifying to the biggest piece before comparing is exactly how you read the running time of real code.

Problem 2-10 BIG OH

IN PLAIN WORDS

Show that $n^3 - 3n^2 - n + 1$ grows at the same rate as n^3 — that is, it is $\Theta(n^3)$.

$\Theta(n^3)$ means “trapped between two constant copies of n^3 once n is big”. The clean trick: divide the whole thing by n^3 and watch it settle.

$$(n^3 - 3n^2 - n + 1) / n^3 = 1 - 3/n - 1/n^2 + 1/n^3$$

Each little fraction ($3/n$, $1/n^2$, $1/n^3$) shrinks to 0 as n grows, so the whole ratio settles near **1**. That means the expression is basically n^3 for large n — caught between, say, $0.5\ n^3$ and $1 \cdot n^3$. Quick check at $n=10$: the value is $1000 - 300 - 10 + 1 = 691$, and indeed $500 \le 691 \le 1000$. Two constant copies of n^3 trap it — that is the definition of Θ.

> ANSWER
>
> It is $\Theta(n^3)$.

WHY IT MATTERS

“Divide by the top term and watch it settle near 1” is the one move that proves any polynomial matches its highest power.

Problem 2-11 BIG OH

IN PLAIN WORDS

Show that n^2 has 2^n as a ceiling — that is, $n^2 = O(2^n)$.

This is the headline ladder fact: an exponential eventually overtakes any polynomial and never looks back. To prove it we just need *one* constant c and a starting point n_0 so that $n^2 \le c \cdot 2^n$ from there on. Watch the race:

N	1	2	3	4	5	6	8	10
n^2	1	4	9	16	25	36	64	100
2^n	2	4	8	16	32	64	256	1024

At $n=3$ the polynomial briefly leads ($9 > 8$), but from **n=4** onward 2^n catches up and then pulls away for good. At $n=10$ it is already ten times bigger, and the gap only grows. So with $c = 1$ and $n_0 = 4$ we have $n^2 \le 2^n$ for all $n \ge 4$ — exactly what a ceiling needs.

> ANSWER
>
> $n^2 = O(2^n)$, shown by $c = 1$, $n_0 = 4$.

WHY IT MATTERS

To prove an O bound you only need one c and one starting point — you do not have to win the race at the very beginning.

Problem 2-12 BIG OH

IN PLAIN WORDS

For each pair, find one positive number c so that $f(n) \le c \cdot g(n)$ for all $n > 1$. The tactic every time: replace each small term of f by a copy of its biggest term, then compare with g.

(a) $f = n^2 + n + 1$, $g = 2n^3$. For $n > 1$, both n and 1 are at most n^2, so $f \le 3n^2$. And $3n^2 \le 4n^3 = 2 \cdot (2n^3)$. So **c = 2** works.

(b) $f = n\sqrt{n} + n^2$ (that is $n^{1.5} + n^2$), $g = n^2$. For $n > 1$, $n^{1.5} \le n^2$, so $f \le n^2 + n^2 = 2n^2$. So **c = 2**.

(c) $f = n^2 - n + 1$, $g = n^2/2$. Since $-n + 1 \le 0$ for $n \ge 1$, we get $f \le n^2 = 2 \cdot (n^2/2)$. So **c = 2**.

Many other constants would also work — you only need to show one.

> ANSWER
>
> $c = 2$ works for all three: (a), (b), and (c).

WHY IT MATTERS

It turns the abstract phrase “there exists a constant c” into a concrete, do-able hunt: bound every term, then pick a c.

Problem 2-13 BIG OH

IN PLAIN WORDS

The **sum rule**. If f_1 has ceiling g_1 and f_2 has ceiling g_2, show that adding them keeps a ceiling: $f_1 + f_2 = O(g_1 + g_2)$.

Remember what a ceiling means: $f = O(g)$ is just "$f \le c \cdot g$ for big n". So we start with two facts:

$$f_1 \le c_1 \cdot g_1 \quad \text{and} \quad f_2 \le c_2 \cdot g_2$$

Add them straight down: $f_1 + f_2 \le c_1 g_1 + c_2 g_2$. Now let c be the bigger of c_1 and c_2. Then the right side is at most $c(g_1 + g_2)$. Done — the sum has ceiling $g_1 + g_2$.

> ANSWER
>
> True: $f_1 + f_2 = O(g_1 + g_2)$.

WHY IT MATTERS

This is the rule that lets you analyse two code blocks one after another and just add their costs (then keep the bigger).

Problem 2-14 BIG OH

IN PLAIN WORDS

The same two rules (sum and product) but with the **floor** Ω instead of the ceiling O. Show that if $f_1 = \Omega(g_1)$ and $f_2 = \Omega(g_2)$, then adding keeps a floor and multiplying keeps a floor.

It is the exact same argument as 2-13 and 2-15, just with the inequalities flipped. A floor means $f \geq c \cdot g$ for big n. So:

- **Sum:** add $f_1 \geq c_1 g_1$ and $f_2 \geq c_2 g_2$; take the smaller of the two c's, and you get a floor on the sum.
- **Product:** multiply the two (everything positive) to get a floor on the product, with constant $c_1 c_2$.

And since Θ is just a ceiling and a floor together, both rules automatically hold for Θ too.

> ANSWER
>
> True: the sum rule and product rule both hold for Ω (and therefore for Θ).

WHY IT MATTERS

Floors combine just like ceilings, so “same rate” (Θ) survives adding and multiplying too.

Problem 2-15 BIG OH

IN PLAIN WORDS

The **product rule**. If f_1 has ceiling g_1 and f_2 has ceiling g_2, show that multiplying them keeps a ceiling: $f_1 \cdot f_2 = O(g_1 \cdot g_2)$.

Start with the same two facts (all quantities positive):

$$f_1 \le c_1 \cdot g_1 \quad \text{and} \quad f_2 \le c_2 \cdot g_2$$

Multiply them together: $f_1 \cdot f_2 \le (c_1\, c_2) \cdot g_1 \cdot g_2$. The single constant $c_1\, c_2$ does the whole job, so the product has ceiling $g_1 \cdot g_2$.

> ANSWER
>
> True: $f_1 \cdot f_2 = O(g_1 \cdot g_2)$.

WHY IT MATTERS

This is why a loop that runs $O(g_1)$ times doing $O(g_2)$ work each pass costs the product $O(g_1\, g_2)$ — the rule behind nested loops.

Problem 2-16 BIG OH

IN PLAIN WORDS

Prove that any polynomial of degree k — something like $a_k n^k + \dots + a_1 n + a_0$ — is $O(n^k)$. In short: the highest power wins.

This is the formal version of “keep the highest power”. Take any single term $a_i\, n^i$ where $i \le k$. Once $n \ge 1$, a lower power of n is no bigger than the top power n^k, so $|a_i\, n^i| \le |a_i| \cdot n^k$. Add up all $k+1$ terms:

$$|a_k n^k + \dots + a_0| \le (|a_k| + \dots + |a_0|) \cdot n^k$$

The bracket $c = |a_k| + \dots + |a_0|$ is just a fixed number — it does not depend on n. So the whole polynomial is at most $c \cdot n^k$, which is exactly $O(n^k)$.

> ANSWER
>
> Any degree-k polynomial $a_k n^k + \dots + a_0$ is $O(n^k)$, with constant $c = |a_k| + \dots + |a_0|$.

WHY IT MATTERS

It justifies the habit you already have: read off a polynomial's running time from its biggest term alone.

Problem 2-17 BIG OH

IN PLAIN WORDS

Show that shifting n by a constant does not change the growth rate: $(n + a)^b = \Theta(n^b)$ for any fixed a and any power $b > 0$.

Intuition first: when n is huge, adding a fixed number a barely nudges it. $1{,}000{,}000 + 7$ is still, for growth purposes, a million. So $(n + a)$ and n grow at the same rate, and raising both to the same power b keeps them matched. To confirm, look at the ratio:

$$(n + a)^b \,/\, n^b = (1 + a/n)^b \to 1 \quad \text{as } n \text{ grows}$$

Because a/n shrinks to 0, the ratio settles at 1. So for big n the expression stays trapped between two fixed constants (say $\frac{1}{2}\, n^b$ and $2\, n^b$) — the sandwich that means Θ.

> ANSWER
>
> $(n + a)^b = \Theta(n^b)$.

WHY IT MATTERS

It lets you drop “+1” and “−3” kinds of shifts inside a running-time expression without a second thought.

Problem 2-18 BIG OH

IN PLAIN WORDS

List these functions from slowest-growing to fastest-growing, and mark any that grow at the same rate (a tie, Θ). This is Skiena's own set of 16 functions:

n 2^n $n \lg n$ $\ln n$ $n - n^3 + 7n^5$ $\lg n$ $\sqrt{n}$ e^n $n^2 + \lg n$ n n^2 2^{n-1} $\lg \lg n$ n^3 $(\lg n)^2$ $n!$ $n^{1+\varepsilon}$ $(0<\varepsilon<1)$

First, tidy each one into its shape:

- $n - n^3 + 7n^5$ → keep the top term, n^5.
- $n^2 + \lg n$ → n^2 (the $\lg n$ is a speck next to n^2) — so it **ties** with plain n^2.
- $\ln n$ and $\lg n$ differ only by a constant factor → they **tie** (both Θ(log n)).
- $2^{n-1} = \frac{1}{2} \cdot 2^n$ → ties with 2^n.
- e^n is *above* 2^n, because $e > 2$ (so $e^n/2^n = (e/2)^n \to \infty$).
- $n^{1+\varepsilon}$ with $0<\varepsilon<1$ sits between $n \lg n$ and n^2 (a real power above n, but below n^2).

Now walk up the ladder. The only surprise to watch: any real power of n (even $\sqrt{n}$) beats every power of a logarithm.

> ANSWER
>
> Slowest → fastest (braces mark ties):
>
> $\lg \lg n \ll \{\ln n = \lg n\} \ll (\lg n)^2 \ll \sqrt{n} \ll n \ll n \lg n \ll n^{1+\varepsilon} \ll \{n^2 = n^2 + \lg n\} \ll n^3 \ll (n - n^3 + 7n^5) \ll \{2^{n-1} = 2^n\} \ll e^n \ll n!$

WHY IT MATTERS

Real problems hand you messy expressions; the skill is to strip each to its dominant term first, then read its rung — spotting the ties is part of the answer.

Problem 2-19 BIG OH

IN PLAIN WORDS

Same task, Skiena's trickier set of 18 functions — sort slowest to fastest, mark ties. Two functions here are sneaky: one *shrinks* to zero, and one is a constant.

$\sqrt{n}$ n 2^n $n \log n$ $n - n^3 + 7n^5$ $n^2 + \log n$ n^2 n^3 $\log n$ $n^{1/3} + \log n$ $(\log n)^2$ $n!$ $\ln n$ $n/\log n$ $\log \log n$ $(1/3)^n$ $(3/2)^n$ 6

Tidy the tricky ones first:

- $(1/3)^n$ → a fraction to a growing power *shrinks toward 0*. It is the **smallest** of all.
- 6 → a constant, Θ(1); above anything that goes to 0, below anything that grows.
- $n - n^3 + 7n^5$ → n^5; $n^2 + \log n$ → n^2 (ties with n^2); $n^{1/3} + \log n$ → $n^{1/3}$.
- $\log n$ and $\ln n$ tie (Θ(log n)). $n/\log n$ is just below n (a shade under linear, but far above $\sqrt{n}$).
- $(3/2)^n$ is below 2^n (smaller base), and both beat every polynomial.

The one easy-to-slip pair: $(\log n)^2$ versus $n^{1/3}$. Squaring a logarithm still leaves a logarithm, and *any* real power of n (even the small power 1/3) eventually beats any power of $\log n$. So $(\log n)^2 < n^{1/3}$ — even though for small n the log looks bigger.

> ANSWER
>
> Slowest → fastest (braces mark ties):
>
> $(1/3)^n \ll 6 \ll \log \log n \ll \{\log n = \ln n\} \ll (\log n)^2 \ll n^{1/3} \ll \sqrt{n} \ll n/\log n \ll n \ll n \log n \ll \{n^2 = n^2 + \log n\} \ll n^3 \ll (n - n^3 + 7n^5) \ll (3/2)^n \ll 2^n \ll n!$

WHY IT MATTERS

The pitfalls here — a function that shrinks, a constant, and a power of n that only overtakes a power of $\log$ much later — are exactly the ones that trip people up in interviews and exams.

Problem 2-20 BIG OH

IN PLAIN WORDS

For each condition, either give two functions `f` and `g` that satisfy it, or explain why it is impossible. Remember: little-o means “strictly slower”, Θ means “same rate”, O means “ceiling”, Ω means “floor”.

(a) Want `f = o(g)` (strictly slower) and `f ≠ Θ(g)` (not the same rate). **Easy.** Take `f = n`, `g = n²`. n is strictly slower than n², and they are clearly not the same rate. In fact “strictly slower” always rules out “same rate”, so any little-o pair works.

(b) Want `f = Θ(g)` and `f = o(g)` at once. **Impossible.** Θ says “exactly the same rate”; little-o says “strictly slower”. Nothing can be both exactly as fast and strictly slower.

(c) Want `f = Θ(g)` but `f ≠ O(g)`. **Impossible.** Θ is a ceiling and a floor together, so `f = Θ(g)` already includes `f = O(g)`. You cannot keep the first and drop the second.

(d) Want `f = Ω(g)` (floor) but `f ≠ O(g)` (no ceiling). **Easy.** Take `f = n²`, `g = n`. n² grows at least as fast as n (floor holds), but it is not capped by n (no ceiling). That is the picture of a strict lower bound.

> ANSWER
>
> (a) possible, e.g. `f = n, g = n²` · (b) impossible · (c) impossible · (d) possible, e.g. `f = n², g = n`.

WHY IT MATTERS

It makes you feel how the five words fit together — which combinations are contradictions and which are everyday examples.

Problem 2-21 BIG OH

IN PLAIN WORDS

True or false for each claim, with a one-line reason from the ladder. Recipe: simplify each side to its biggest piece, then check whether the left really is a ceiling of the right.

	CLAIM	VERDICT	PLAIN REASON
(a)	$2n^2 + 1 = O(n^2)$	True	constant factor plus a tiny term; still n^2
(b)	$\sqrt{n} = O(\log n)$	False	$\sqrt{n}$ grows faster than $\log n$, not slower
(c)	$\log n = O(\sqrt{n})$	True	a log sits below any power of n
(d)	$n^2(1 + \sqrt{n}) = O(n^2 \log n)$	False	left side is about $n^2 \cdot \sqrt{n} = n^{2.5}$, above $n^2 \log n$
(e)	$3n^2 + \sqrt{n} = O(n^2)$	True	$\sqrt{n}$ is a tiny term; n^2 dominates
(f)	$\sqrt{n} \log n = O(n)$	True	$\sqrt{n} \cdot \log n$ grows slower than $\sqrt{n} \cdot \sqrt{n} = n$
(g)	$\log n = O(n^{-1/2})$	False	$\log n$ grows to infinity while $n^{-1/2} = 1/\sqrt{n}$ shrinks to 0

ANSWER

(a) True · (b) False · (c) True · (d) False · (e) True · (f) True · (g) False.

WHY IT MATTERS

These are the exact patterns (drop small terms, drop constants, compare rungs) you will apply every time you eyeball a running time.

Problem 2-22 BIG OH

IN PLAIN WORDS

State the tightest single relationship of f to g: is f a ceiling (O), a floor (Ω), the same rate (Θ), or none? Simplify each to its biggest piece first.

(a) $f = n^2 + 3n + 4$, $g = 6n + 7$. f is quadratic, g is linear, so f grows faster: $f = \Omega(g)$ (and definitely not a ceiling).

(b) $f = n\sqrt{n} = n^{1.5}$, $g = n^2 - n$. g is basically n^2, above $n^{1.5}$, so f is the smaller one: $f = O(g)$.

(c) $f = 2^n - n^2$, $g = n^4 + n^2$. The 2^n completely swamps the $-n^2$ (a rounding error beside it), so f grows like 2^n — an exponential, above the polynomial g: $f = \Omega(g)$.

> ANSWER
>
> (a) Ω · (b) O · (c) Ω.

WHY IT MATTERS

It trains you to ignore the noise (small and subtracted terms) and judge by the one dominant piece.

Problem 2-23 BIG OH

IN PLAIN WORDS

A yes/no quiz about what Big-O really promises. Key idea: O is only a *ceiling* on the worst case — it never stops an algorithm from being faster. Θ on the worst case is stronger: it says the worst case really does reach that height.

(a) Worst case $O(n^2)$ — can it be $O(n)$ on *some* inputs? **Yes.** Some inputs (like a best case) can finish far faster; the ceiling only caps the slowest.

(b) Worst case $O(n^2)$ — can it be $O(n)$ on *all* inputs? **Yes.** $O(n^2)$ is only an upper bound; an algorithm that is actually linear everywhere is still (loosely) $O(n^2)$. O does not claim the bound is tight.

(c) Worst case $\Theta(n^2)$ — can it be $O(n)$ on *some* inputs? **Yes.** Θ pins the *worst* case at quadratic, but the best case (some inputs) can still be linear.

(d) Worst case $\Theta(n^2)$ — can it be $O(n)$ on *all* inputs? **No.** A $\Theta(n^2)$ worst case means some inputs genuinely force quadratic work, so not every input can be linear.

(e) Is $f(n) = \Theta(n^2)$ when $f = 100n^2$ for even n and $f = 20n^2 - n \cdot \log^2 n$ for odd n? **Yes.** Either way, every value sits between constant copies of n^2 (roughly between $19n^2$ and $100n^2$ for large n), so the whole function is $\Theta(n^2)$ no matter which branch it takes.

> ANSWER
>
> (a) yes · (b) yes · (c) yes · (d) no · (e) yes.

WHY IT MATTERS

It fixes the most common misreading: O is a promise of “no worse than”, not “always exactly this” — only Θ makes the tight claim.

Problem 2-24 BIG OH

IN PLAIN WORDS

Yes, no, or can't-tell for each claim about exponentials and their logs. Handy fact: $\log(a^n) = n \cdot \log a$ — taking a log turns an exponential into a plain multiple of n.

(a) Is $3^n = O(2^n)$? **No.** The ratio $3^n / 2^n = (3/2)^n = 1.5^n$ races off to infinity, so no constant can cap it. Different bases are not a constant factor apart.

(b) Is $\log(3^n) = O(\log(2^n))$? **Yes.** Take logs: $\log(3^n) = n \log 3$ and $\log(2^n) = n \log 2$. Their ratio is the fixed number $\log 3 / \log 2 \approx 1.585$ — a constant factor — so each is a ceiling of the other. Taking the log tamed the exponential into a linear function of n.

(c) Is $3^n = \Omega(2^n)$? **Yes.** A floor asks whether 3^n grows *at least* as fast as 2^n — and it grows much faster, so certainly at least as fast.

(d) Is $\log(3^n) = \Omega(\log(2^n))$? **Yes.** Same constant-ratio picture as (b): $n \log 3$ versus $n \log 2$. A constant factor apart means each is both a ceiling and a floor of the other.

> ANSWER
>
> (a) no · (b) yes · (c) yes · (d) yes.

WHY IT MATTERS

It shows the magic of logs: two exponentials that are worlds apart become mere constant-factor neighbours once you take their logarithms.

Ordering & Summation Growth

Problem 2-25 SUMMATION ASYMPTOTICS

IN PLAIN WORDS

We add up four different lists of numbers, with `i` counting from 1 up to n. For each list, we just want to know how fast the total grows as n gets big. "Grows at the same rate as g" is written Θ(g).

A Σ (sigma) just means "add these up".

(a) Add up 1/i: that is 1/1 + 1/2 + 1/3 + ... + 1/n. Each new piece is tiny, so the total creeps up very slowly. It turns out to stay close to `log n` forever (the gap between them settles near 0.577 and stops moving). So the total grows at the same rate as `log n`.

(b) Add up ⌈1/i⌉: the ⌈ ⌉ brackets mean "round up to the next whole number". For any i of 1 or more, 1/i is a number between 0 and 1, so rounding it up always gives exactly 1. Adding 1 to itself n times gives n. So the total grows like `n`.

(c) Add up log i: that is log 1 + log 2 + ... + log n. A handy fact: adding logs is the same as taking the log of the product. So this equals log(1 · 2 · 3 · ... · n) = log(n!). See part (d): it grows like `n log n`.

(d) log(n!): here n! means 1 · 2 · ... · n. At least half of those factors are n/2 or bigger, so n! is at least (n/2) multiplied by itself n/2 times — that makes log(n!) at least about (n/2)·log(n/2), which is on the order of n log n. And n! is at most n multiplied by itself n times, so log(n!) is at most n log n. Squeezed from both sides, it grows like `n log n`.

> ANSWER
>
> (a) Θ(log n) · (b) Θ(n) · (c) Θ(n log n) · (d) Θ(n log n).

WHY IT MATTERS

Knowing the growth rate of a sum lets you predict how slow a loop will be without adding up every single term.

Problem 2-26 PUT FUNCTIONS IN ORDER

IN PLAIN WORDS

We have four formulas. Line them up from the slowest-growing to the fastest-growing as n gets big.

The four are:

- f1 = $n^2 \log n$
- f2 = $n (\log n)^2$
- f3 = $2^0 + 2^1 + ... + 2^n$ (add up the powers of two)
- f4 = log of that same sum

First, tidy up the two that hide a sum. A sum of powers that keep doubling is basically its last term. So f3 = 1 + 2 + 4 + ... + 2^n adds up to just over 2^n — it grows like 2^n. And f4 is the log of that. Since the log of 2^n is about n, f4 grows like n.

Now compare the simple shapes: n, $n(\log n)^2$, $n^2 \log n$, 2^n. To see which of f2 and f1 is bigger, divide both by n log n: f2 becomes log n, f1 becomes n. Since n is far bigger than log n, f1 beats f2.

> ANSWER
>
> f4 ($\approx n$) < f2 ($n \log^2 n$) < f1 ($n^2 \log n$) < f3 (2^n).

WHY IT MATTERS

Sorting formulas by growth rate is how you tell at a glance which algorithm will win on big inputs.

Problem 2-27 ORDER FUNCTIONS, NOTE TIES

IN PLAIN WORDS

Another four formulas to line up slowest to fastest — and this time, say if any two grow at the same rate.

The four are:

- f1 = √1 + √2 + ... + √n (add up the square roots)
- f2 = $(\sqrt{n})^{\log n}$
- f3 = $n^{\sqrt{(\log n)}}$
- f4 = $12 \cdot n^{1.5} + 4n$

Simplify each:

f1: adding up √1, √2, ..., √n grows like $n^{1.5}$ (about two-thirds of $n^{1.5}$).

f4: keep the biggest piece, $12 \cdot n^{1.5}$, and drop the constant — it is also $n^{1.5}$. So f1 and f4 grow at the same rate (a tie).

f2: this rewrites as n raised to the power (log n)/2. The power itself grows with n, so it beats any fixed power of n.

f3: this is n raised to the power √(log n). That power grows too, so it also beats any fixed power — but slower than f2, because (log n)/2 eventually gets much bigger than √(log n).

> ANSWER
>
> f1 = f4 (both $n^{1.5}$) < f3 ($n^{\sqrt{\log n}}$) < f2 ($n^{(\log n)/2}$).

WHY IT MATTERS

A fixed power like $n^{1.5}$ always loses to n raised to a power that itself keeps growing.

Problem 2-28 SIMPLIFY EACH SUM

IN PLAIN WORDS

For each of three sums (add up the terms for i = 1 to n), give the simplest formula it grows like.

Two easy facts do all the work:

- If you add up n terms whose biggest is a power like i^k, the total grows like n^{k+1} (roughly n copies of the top term).
- A sum where each term is a fixed multiple of the last (like 4^i, which keeps quadrupling) is basically its final term.

(a) add up $3i^4 + 2i^3 - 19i + 20$: the biggest piece is i^4, and adding up i^4 grows like n^5. So $\Theta(n^5)$.

(b) add up $3 \cdot 4^i + 2 \cdot 3^i - i^{19} + 20$: the 4^i term quadruples each step and outgrows everything. That sum is basically its last term, 4^n. So $\Theta(4^n)$.

(c) add up $5i + 3 \cdot 2^i$: the 2^i term doubles each step and beats the plain 5i. The sum is basically its last term. So $\Theta(2^n)$.

> ANSWER
>
> (a) $\Theta(n^5)$ · (b) $\Theta(4^n)$ · (c) $\Theta(2^n)$.

WHY IT MATTERS

Most messy sums collapse to one simple term, so you rarely need to compute the whole thing.

Problem 2-29 A GEOMETRIC SUM

IN PLAIN WORDS

Let S be the sum $3^1 + 3^2 + ... + 3^n$ (add up the powers of three). We are shown three guesses for its growth rate and asked which are true.

The three guesses are: (a) S grows like 3^{n-1}, (b) S grows like 3^n, (c) S grows like 3^{n+1}.

A sum of powers that keep tripling is basically its last term. Working it out exactly, $S = (3^{n+1} - 3) / 2$, which is about $3^{n+1}/2$. Either way, the last term rules, so S grows like 3^n.

Here is the trick: 3^{n-1}, 3^n, and 3^{n+1} only differ by a factor of 3 up or down. And a plain factor of 3 does not change a growth rate — Θ ignores constant multipliers. So all three name the very same growth class.

> ANSWER
>
> (a), (b) and (c) are all true — they name the same growth rate up to a constant factor.

WHY IT MATTERS

Constant factors vanish in Θ, so several different-looking answers can all be correct.

Problem 2-30 KEEP THE FASTEST TERM

IN PLAIN WORDS

For each formula f, give the simplest thing it grows like. The rule: keep the biggest piece and drop its constant.

(a) $f = 1000 \cdot 2^n + 4^n$: note 4^n equals $(2^n)^2$, so it dwarfs 2^n no matter how big the 1000 is. Keep 4^n. So $\theta(4^n)$.

(b) $f = n + n \log n + \sqrt{n}$: of the three, n log n grows fastest. So $\theta(n \log n)$.

(c) $f = \log(n^{20}) + (\log n)^{10}$: the first piece is just 20 log n (a constant times log n). The second is a much higher power of log n and wins. So $\theta((\log n)^{10})$.

(d) $f = (0.99)^n + n^{100}$: a base below 1 means $(0.99)^n$ shrinks toward 0 as n grows, so it adds nothing. Only n^{100} matters. So $\theta(n^{100})$.

> ANSWER
>
> (a) $\Theta(4^n)$ · (b) $\Theta(n \log n)$ · (c) $\Theta((\log n)^{10})$ · (d) $\Theta(n^{100})$.

WHY IT MATTERS

For big n, only the fastest-growing piece counts — the rest is noise.

Problem 2-31 WHICH RELATIONS HOLD

IN PLAIN WORDS

For each pair (A, B), say which growth relationships hold. Quick gloss: O means "A is no faster than B", o (little-oh) means "A is strictly slower than B", Ω means "A is no slower", ω means "A is strictly faster", and Θ means "same rate". If A is strictly slower than B, then both O and the stronger o hold.

(a) $A = n^{100}$, $B = 2^n$: a polynomial is strictly slower than an exponential. So O and o.

(b) $A = (\log n)^{12}$, $B = \sqrt{n}$: any power of a log is strictly slower than any power of n. So O and o.

(c) $A = \sqrt{n}$, $B = n^{\cos(\pi n/8)}$: the cos part swings forever between −1 and +1, so B keeps swinging from 1/n up to n and back. Sometimes B is way above A, sometimes way below — no single relationship holds for all big n. So **none** apply (a genuine "can't compare" case).

(d) $A = 10^n$, $B = 100^n$: since 100 > 10, the ratio 0.1^n shrinks to 0, so A is strictly slower. So O and o.

(e) $A = n^{\log n}$, $B = (\log n)^n$: compare their logs: $\log A = (\log n)^2$, while $\log B = n \cdot \log \log n$. The second grows far faster (it has a factor of n), so B beats A. So O and o.

(f) $A = \log(n!)$, $B = n \log n$: from Problem 2-25, $\log(n!)$ grows at the same rate as $n \log n$. So Θ (and therefore also O and Ω, but not the strict little versions).

> ANSWER
>
> (a) O, o · (b) O, o · (c) none · (d) O, o · (e) O, o · (f) Θ (with O and Ω).

WHY IT MATTERS

These five relations are the vocabulary for saying exactly how two running times compare.

Summations

Problem 2-32 ALTERNATING SUM OF SQUARES

IN PLAIN WORDS

Show that the plus-minus sum of squares $1^2 - 2^2 + 3^2 - 4^2 + \ldots$ equals (with the right sign) $k(k+1)/2$.

We add the squares 1, 4, 9, 16, ... but flip the sign on every second one: plus, minus, plus, minus. The claim is that the messy total is always just $k(k+1)/2$ (the plain "add 1 up to k" number), with a plus sign when k is odd and a minus sign when k is even. (That is all the $(-1)^{k-1}$ factor does: +1 for odd k, −1 for even k.)

Check small cases and watch both sides line up:

K	LEFT (THE PLUS-MINUS SUM)	RIGHT (SIGN)·K(K+1)/2
1	1 = 1	+ 1·2/2 = 1
2	1 − 4 = −3	− 2·3/2 = −3
3	1 − 4 + 9 = 6	+ 3·4/2 = 6
4	1 − 4 + 9 − 16 = −10	− 4·5/2 = −10
5	... + 25 = 15	+ 5·6/2 = 15

Why it works: pair each minus with the plus just before it: $(1^2 - 2^2)$, $(3^2 - 4^2)$, and so on. Each such pair equals the negative of the two numbers added together. So the whole plus-minus sum of squares turns into a plain running total of 1, 2, 3, ... (carrying a sign) — and a running total of 1 up to k is exactly $k(k+1)/2$. The final sign is whatever the last term had: plus for odd k, minus for even k.

> ANSWER
>
> The identity holds: $1^2 - 2^2 + \ldots + (-1)^{k-1} k^2 = (-1)^{k-1} \cdot k(k+1)/2$ (checked for k = 1..20 and explained by pairing consecutive terms).

WHY IT MATTERS

Pairing terms turns a scary alternating sum into the friendly triangle-number formula.

Problem 2-33 A SELF-REFERENTIAL TRIANGLE

IN PLAIN WORDS

Build a triangle where each entry is the sum of the three entries above it (above-left, above, above-right; treat any missing neighbour as 0). Find a formula for the sum of the entries in row i.

The triangle starts like this:

ROW	ENTRIES
1	1
2	1 1 1
3	1 2 3 2 1
4	1 3 6 7 6 3 1

Add up each row: row 1 gives 1, row 2 gives 3, row 3 gives 9, row 4 gives 27. That is 1, 3, 9, 27 — the powers of three. So the answer looks like 3^{i-1}.

Why: ask where each entry lands in the row below. Every entry is one of the "three above" for exactly three spots underneath it — the slot below-left, the slot directly below, and the slot below-right. So each entry passes its full value down three times. Add that across the whole row: (sum of next row) = 3 × (sum of this row). The total triples at every step. Row 1 starts at 1, so row i is 1 tripled i−1 times, which is 3^{i-1}.

> ANSWER
>
> Row i sums to 3^{i-1}, because each entry feeds into three entries below it, so the total triples with every new row.

WHY IT MATTERS

Counting how many times each value gets reused is a reliable trick for finding a sum's formula.

Problem 2-34 TWELVE DAYS OF CHRISTMAS

IN PLAIN WORDS

In the song, on day 1 you get 1 gift, on day 2 you get 2 new gifts plus the 1 from before, and so on — each day you get that day's gifts on top of all the earlier ones. If Christmas lasts n days, how many presents arrive in total?

One day at a time: on day d the true love brings 1 + 2 + ... + d presents (that day's new gift plus everything repeated). That is the "add 1 up to d" number, so day d brings d(d+1)/2 presents.

Add up all the days: the grand total is the sum of each day's haul, Σ (for d = 1 to n) of d(d+1)/2. A small table:

DAY N	PRESENTS THAT DAY = N(N+1)/2	TOTAL SO FAR
1	1	1
2	3	4
3	6	10
4	10	20

Those running totals 1, 4, 10, 20 have a tidy formula: n(n+1)(n+2)/6. Check it: for n = 4 that is 4·5·6/6 = 20. For the real song, n = 12: 12·13·14/6 = 2184/6 = 364.

> ANSWER
>
> Total(n) = $\Sigma_{d=1}^{n}$ d(d+1)/2 = n(n+1)(n+2)/6, which for n = 12 is 364 presents. The growth is $\Theta(n^3)$.

WHY IT MATTERS

A stack of triangle numbers has its own neat closed form, so you never have to add all n days by hand.

Problem 2-35 HOW MANY TIMES DOES THE LOOP RUN

IN PLAIN WORDS

An outer loop runs i from 1 to n. For each i, an inner loop runs j from i up to 2i, printing once each time. Let T(n) be the total number of prints. (a) Write T(n) as a sum. (b) Simplify it.

One pass of the outer loop: fix i. The inner loop runs j from i up to and including 2i. How many whole numbers is that? (2i − i) + 1 = i + 1. (The "+1" is because both ends count.) Quick check with i = 3: j is 3, 4, 5, 6 — four values, and i + 1 = 4. Good.

(a) Add up every outer pass: the outer loop does this for i = 1, 2, ..., n, so add up (i + 1) for each. As a sum: $T(n) = \Sigma_{i=1}^{n} (i + 1)$ (that Σ just means "add these up").

(b) Simplify: split into two easier sums:

T(n) = (1 + 2 + ... + n) + (1 + 1 + ... + 1). The first bracket is the "add 1 up to n" number, n(n+1)/2. The second is 1 added n times, which is n. So T(n) = n(n+1)/2 + n = (n² + 3n)/2.

The biggest piece is n², so this grows like Θ(n²) — the usual two-nested-loops behaviour.

> ANSWER
>
> (a) $T(n) = \Sigma_{i=1}^{n} (i + 1)$. (b) T(n) = n(n+1)/2 + n = (n² + 3n)/2, which is Θ(n²).

WHY IT MATTERS

Turning a loop's step count into a formula is exactly how you find its running time.

Problem 2-36 NESTED SUMMATIONS

IN PLAIN WORDS

Some code prints from inside three nested loops (take `n` even). (a) Write the running time `T(n)` as three nested sums. (b) Simplify the sum and show the work.

The loops run: `i` from 1 to `n/2`, then `j` from `i` to `n−i`, then `k` from 1 to `j`, printing once at the centre.

(a) The nested-sum form (each loop becomes one ∑, the printed line is the "1" being counted):

$$T(n) = \sum_{i=1}^{n/2} \sum_{j=i}^{n-i} \sum_{k=1}^{j} 1$$

(b) Simplify. The innermost $\sum_{k=1}^{j} 1$ is just `j`. So

$$T(n) = \sum_{i=1}^{n/2} \sum_{j=i}^{n-i} j$$

The inner sum adds the whole numbers from `i` up to `n−i`. Using "sum up to b minus sum up to a−1", that inner sum equals `[(n−i)(n−i+1) − (i−1)i] / 2`. Writing `n = 2m` and adding these up over `i = 1 .. m`, everything collapses to a strikingly clean result:

$$T(n) = m^3 = (n/2)^3 = n^3 / 8$$

Check: for `n = 4` (`m = 2`) this predicts `8`, and counting the prints by hand gives 8. For `n = 10` it predicts `125`, which also matches a direct count.

> ANSWER
>
> (a) $T(n) = \sum_{i=1}^{n/2} \sum_{j=i}^{n-i} \sum_{k=1}^{j} 1$. (b) It simplifies exactly to $T(n) = n^3/8$ for even n, so $T(n) = \Theta(n^3)$.

WHY IT MATTERS

Skiena asks you to *simplify*, not just name the class — and the exact answer $n^3/8$ is far more satisfying (and useful) than a bare $\Theta(n^3)$.

Problem 2-37 MULTIPLY BY REPEATED ADDITION

IN PLAIN WORDS

You first learned multiplication as repeated adding: 5 × 4 = 5 + 5 + 5 + 5. How long does it take to multiply two n-digit numbers (in base b) this way?

To do x × y by repeated addition, we add x to a running total y times. So the cost depends on how big y can get. An n-digit number in base b is at most about b^n (in base 10 a 3-digit number reaches about $10^3 = 1000$; in base 2 a 3-bit number reaches about $2^3 = 8$). So y can force about b^n additions.

Each addition is of two n-digit numbers, which costs about n steps (add digit by digit, carrying). Multiply the two together: about b^n additions × about n per addition gives `O(n · b^n)`.

DIGITS N	LARGEST VALUE ≈ 10^n	ADDITIONS NEEDED
1	9	up to 9
3	999	up to ~1 000
6	999 999	up to ~1 000 000
10	~10^{10}	up to ~10 billion
20	~10^{20}	hopelessly many

That b^n is exponential in n — runaway growth. Fine for 5 × 4, hopeless for real numbers.

> ANSWER
>
> $O(n \cdot b^n)$ — exponential in the number of digits.

WHY IT MATTERS

A method can be perfectly correct and still far too slow to ever use.

Problem 2-38 LONG MULTIPLICATION

IN PLAIN WORDS

The grade-school way: multiply digit by digit, line up the partial rows, and add. How long does it take to multiply two n-digit numbers (fixed base)?

Long multiplication pairs every digit of one number with every digit of the other. With n digits each, that is $n \times n = n^2$ single-digit multiplications. Then we line up those partial rows and add them — that is another n^2-ish of work (about n rows, each up to about n digits wide).

So: n^2 single-digit products + about n^2 additions to combine gives `O(n²)`.

Stand the two methods side by side:

METHOD	CLASS	TWO 20-DIGIT NUMBERS
2-37 repeated addition	$O(n \cdot b^n)$	about 10^{20} steps — impossible
2-38 long multiplication	$O(n^2)$	about 400 steps — instant

> ANSWER
>
> $O(n^2)$.

WHY IT MATTERS

Picking a smarter method turns an impossible calculation into an instant one.

Logarithms

Problem 2-39 LOGARITHMS

IN PLAIN WORDS

Prove four rules about logs. Sounds scary, but there is one idea behind all of them: a log is just an exponent. If $\log_2 8 = 3$, that only means $2^3 = 8$. Read every log as "the power you raise the base to."

Keep flipping between the two ways of saying the same thing: "$\log_a x = p$" and "$x = a^p$". That single trick unlocks all four rules.

(a) $\log(x \cdot y) = \log x + \log y$. Say $x = a^p$ and $y = a^q$. When you multiply two powers you add the exponents: $x \cdot y = a^{p+q}$. So the exponent for $x \cdot y$ is $p + q$, which is $\log x + \log y$. (This is the old trick that turned multiplication into addition on slide rules.)

(b) $\log(x^y) = y \cdot \log x$. With $x = a^p$, raising to the power y multiplies the exponent: $x^y = a^{p \cdot y}$. So the exponent is $p \cdot y$, which is $y \cdot \log x$.

(c) change of base: $\log_a x = \log_b x / \log_b a$. Start from $x = a^p$ where $p = \log_a x$. Take log base b of both sides and use rule (b): $\log_b x = p \cdot \log_b a$. Divide across and $p = \log_b x / \log_b a$.

(d) $x^{\log_b y} = y^{\log_b x}$. Two things are equal if their logs match. Take log base b of each side using rule (b): the left gives $\log_b y \cdot \log_b x$, the right gives $\log_b x \cdot \log_b y$. Same product, so the two sides are equal.

Rule (c) is the important one for this course. It says any two bases differ only by a fixed number ($1 / \log_b a$). Big-O throws away fixed multipliers, so *the base of a log never matters for Big-O*. That is why we happily write $O(\log n)$ without ever saying which base.

> ANSWER
>
> All four rules hold. Each is just an exponent rule read through the "log = the exponent" lens. Rule (c) is why the log's base makes no difference in Big-O.

WHY IT MATTERS

It lets us drop the base and write $O(\log n)$ for binary search and every other halving algorithm without fuss.

Problem 2-40 LOGARITHMS

IN PLAIN WORDS

Show that ⌈lg(n+1)⌉ = ⌊lg n⌋ + 1 for every whole number n ≥ 1. Here "lg" means log base 2, and the fancy brackets just mean "round up" (⌈ ⌉) or "round down" (⌊ ⌋).

Both sides are secretly counting the same thing: how many bits n needs to be written in binary, which is the same as how many times you halve n before you reach 1. Let us just make a small table and watch the two columns agree.

N	LG N	⌊LG N⌋ + 1	LG(N+1)	⌈LG(N+1)⌉
1	0.00	1	1.00	1
2	1.00	2	1.58	2
3	1.58	2	2.00	2
4	2.00	3	2.32	3
5	2.32	3	2.58	3
6	2.58	3	2.81	3
7	2.81	3	3.00	3
8	3.00	4	3.17	4

The third column and the last column match on every row. Nice and simple.

> ANSWER
>
> Both sides count the number of bits of n, so ⌈lg(n+1)⌉ = ⌊lg n⌋ + 1 for all n ≥ 1.

WHY IT MATTERS

Two different-looking formulas turn out to be the same "how many halvings" count that shows up all over searching.

Problem 2-41 LOGARITHMS

IN PLAIN WORDS

Show that writing a whole number n ≥ 1 in binary takes exactly ⌊lg n⌋ + 1 bits (⌊ ⌋ means round down).

The biggest number you can write with b bits is all ones: 1 bit reaches 1, 2 bits reach 3, 3 bits reach 7, 8 bits reach 255. So a number n needs exactly b bits when it fits in b bits but not in b−1. In symbols, n sits between 2^{b-1} and 2^b (that is, $2^{b-1} \le n < 2^b$). Taking lg of that, $b-1 \le \lg n < b$, so rounding lg n down gives b−1, and $b = \lfloor \lg n \rfloor + 1$. A tiny table shows it:

N	BINARY	BITS	⌊LG N⌋ + 1
1	1	1	0 + 1 = 1
2	10	2	1 + 1 = 2
4	100	3	2 + 1 = 3
255	11111111	8	7 + 1 = 8
256	100000000	9	8 + 1 = 9

> ANSWER
>
> A whole number n needs ⌊lg n⌋ + 1 bits in binary. This is the same count as problem 2-40.

WHY IT MATTERS

It is exactly why binary search takes about lg n steps: each comparison learns one bit about where your target is.

Problem 2-42 LOGARITHMS

IN PLAIN WORDS

Someone shows you a sort that runs in O(n log √n). But everyone says sorting needs at least about n log n comparisons. Is the person cheating?

No cheating, just a disguise. Simplify the log with rule (b) from problem 2-39. Since √n = $n^{1/2}$:

$$\log \sqrt{n} = \log(n^{1/2}) = \tfrac{1}{2} \cdot \log n$$

So O(n log √n) = O(n · ½ log n) = O(½ · n log n). The ½ is just a fixed multiplier, and Big-O throws fixed multipliers away. So this is O(n log n) — the very same class.

> ANSWER
>
> Nothing is broken. O(n log √n) is exactly the same class as O(n log n), only written in disguise. It does not beat the lower bound; it sits right on it.

WHY IT MATTERS

A good reminder that "log √n" and "½ log n" are the same thing, and constant factors never change the Big-O class.

Interview & Puzzle Problems

Problem 2-43 INTERVIEW

IN PLAIN WORDS

Numbers arrive one at a time, like cars passing a window. You want to keep a fair random handful of k of them, giving every number the same chance of being kept — and you only get one look at each, and you do not even know how many will pass in the end.

This is called **reservoir sampling**. Start with the easiest version, keeping just k = 1.

The rule: when the i-th number arrives (counting 1, 2, 3, …), keep it with probability 1/i; otherwise stick with the one you already hold. So you always keep the 1st. You swap to the 2nd with chance 1/2. You swap to the 3rd with chance 1/3. And so on.

Why is that fair? The very last number, number n, is kept with chance 1/n straight away — good. An earlier number j was kept when it arrived (chance 1/j) and then had to survive every later number choosing not to replace it. Those survival chances multiply out to j/n. Put together, everyone ends up with the same 1/n. Fair.

For a handful of k: keep the first k as your starting "reservoir". When the i-th number arrives (with i bigger than k), keep it with chance k/i; if you keep it, randomly throw out one of the k you were holding to make room. The same argument gives every number the final chance k/n.

```
import random

def reservoir(stream, k):
    keep = []
    for i, item in enumerate(stream):   # i counts from 0; n never needed
        if i < k:
            keep.append(item)           # fill the reservoir first
        else:
            j = random.randint(0, i)    # keep with probability k/(i+1)
            if j < k:
                keep[j] = item          # evict a random held item, inser
    return keep
```

It reads the stream once, stores only k items, and never needs to know the total ahead of time — perfect for a stream too big to hold or of unknown length.

> ANSWER

> Reservoir sampling: hold the first k items; for each later item i, keep it with probability k/i and evict a random current member. One pass, stores only k, and n need not be known in advance.

WHY IT MATTERS

It is the standard way to sample fairly from a stream you cannot store or measure — a real interview favourite.

Problem 2-44 INTERVIEW

IN PLAIN WORDS

You have 1000 items spread over 1000 computers (nodes), and each node holds 3 items. Nodes sometimes die. How should you place things so you lose as little as possible, and roughly how much do you expect to lose if 3 random nodes fail at once?

The core idea is simple: **keep three copies of every item, on three well-spread-out nodes**, and never let two items live on the exact same trio of nodes. An easy way: put item i on nodes i, i+1 and i+2 (wrapping around at the end). Now every item has three homes, every node still holds three items, and each item's trio of homes is different.

An item is only lost if *all three* of its nodes die. If 3 random nodes fail, one item is lost only if those 3 dead nodes are exactly its 3 homes. The chance of that for one item is 1 divided by the number of ways to pick 3 nodes out of 1000 — a huge number, about 166 million. So one item's chance of loss is tiny. Add it up over all 1000 items:

$$\text{expected items lost} \approx 1000 \times 1 / C(1000, 3) \approx 6 \times 10^{-6}$$

That is essentially zero. Almost always you lose nothing at all; in the rare unlucky case you lose about one item.

> ANSWER
>
> Three copies per item, spread over well-separated nodes so no two items share a trio. With 3 random node failures the expected loss is essentially zero (about 6 in a million items).

WHY IT MATTERS

It shows how a little replication plus spreading copies out makes data survive hardware failures almost for free.

Problem 2-45 INTERVIEW

IN PLAIN WORDS

To find the smallest value in an array `A[0..n]` (that is `n+1` numbers), start with `tmp = A[0]`, then scan `A[1], A[2], …, A[n]` and set `tmp = A[i]` whenever `A[i]` is smaller. On a random ordering, how many times, on average, does that assignment `tmp = A[i]` actually run?

Careful about what is counted. Setting up `tmp = A[0]` at the start is the initial value, *not* one of the assignments we are counting. We only count the assignments made *during the scan*, at positions `i = 1 .. n`.

When does position `i` assign? Only when `A[i]` is smaller than everything seen before it — i.e. when `A[i]` is the smallest among the first `i+1` numbers `A[0..i]`. In a random order, each of those `i+1` numbers is equally likely to be the smallest, so the chance is `1/(i+1)`.

Add the chances over the scanned positions `i = 1 .. n`:

$$\text{expected assignments} = \sum_{i=1}^{n} 1/(i+1) = 1/2 + 1/3 + \ldots + 1/(n+1) = H_{n+1} - 1$$

Here $H_m = 1 + 1/2 + \ldots + 1/m$ is the harmonic number, and $H_{n+1} - 1$ is just it with the first term (the 1) removed. It grows about like `ln n` — painfully slowly:

ARRAY SIZE (N+1 ELEMENTS)	EXPECTED ASSIGNMENTS = $H_{N+1} - 1$
n = 10	about 2.02
n = 100	about 4.19
n = 1 000	about 6.49
n = 1 000 000	about 13.4

(A quick simulation over many random shuffles agrees: about 2.0 for `n = 10` and about 3.5 for `n = 50`.) If your array is defined as exactly `n` elements instead, the same argument gives $H_n - 1$.

> ANSWER
>
> The assignment fires at position `i` with probability `1/(i+1)`, so the expected number is $H_{n+1} - 1 = 1/2 + 1/3 + \ldots + 1/(n+1) \approx \ln n$ — that is `O(log n)`. (It is *not* the full H_n: the initial `tmp = A[0]` is not a counted assignment.)

WHY IT MATTERS

A tiny index shift changes the exact answer — and it is a lovely case of very slow, logarithmic growth hiding inside a plain loop.

Problem 2-46 PUZZLE

IN PLAIN WORDS

A 100-floor building. Dropped from some floor and above, a marble breaks; below it, the marble is fine. You want the lowest breaking floor. How few drops do you need (a) with lots of marbles, and (b) with only two?

(a) Lots of marbles → binary search. A broken marble costs you nothing, so you can gamble on the middle floor every time. Drop from 50. If it breaks, the answer is below 50; if not, above. Each drop halves the range, so you need $\lceil \lg 100 \rceil$ = **7 drops** — the same halving trick as binary search.

(b) Only two marbles → you cannot halve. If your first marble breaks at floor 50, you only have one marble left, and you dare not gamble with it — you must walk up one floor at a time from the last safe floor. So a big first jump risks up to 49 more drops. The fix: make the first marble take *shrinking* steps, so that whichever gap it breaks in, the walk-up with the second marble is short.

Drop the first marble at floors 14, 27, 39, 50, 60, 69, 77, 84, 90, 95, 99, 100 — gaps of 14, then 13, then 12, shrinking by one each time.

Why start at 14? If the first marble breaks on drop number d, you have already spent d drops and the gap left to walk is 14 − d floors — always about 14 total. You want the first step k big enough that $k + (k-1) + \dots + 1 = k(k+1)/2$ covers all 100 floors. The smallest k that works is 14, because $14 \times 15 / 2 = 105 \geq 100$, while $13 \times 14 / 2 = 91$ falls short.

> ANSWER
>
> 7 drops with plenty of marbles (binary search); about 14 drops in the worst case with only two, using shrinking gaps. Two marbles cost you the logarithm — you go from lg n up to roughly √n.

WHY IT MATTERS

It shows how few resources (only 2 marbles) forces a slower strategy than clean halving.

Problem 2-47 PUZZLE

IN PLAIN WORDS

Ten bags of coins. Nine bags have real 10-gram coins; one bag has fake 9-gram coins. Your scale shows exact grams. Find the fake bag with just **one** weighing.

The trick is to make each bag leave a different, recognisable fingerprint on one pile. Number the bags 1 to 10. Take 1 coin from bag 1, 2 coins from bag 2, 3 from bag 3, …, 10 from bag 10 — that is 55 coins in all.

If every coin were real, the pile would weigh 55 × 10 = 550 grams. Each fake coin is 1 gram light, and bag k put in exactly k coins. So if bag k is the fake one, the pile reads 550 − k grams. **The number of grams missing is the guilty bag's number.**

```
weights = [10] * 10       # ten bags, all 10-gram coins to start
weights[6] = 9            # bag number 7 secretly holds 9-gram coins

# take (i+1) coins from bag i and weigh them all together
total = sum((i + 1) * weights[i] for i in range(10))
print(550 - total)        # 7  -> the light bag is number 7
```

Weigh 5 grams missing? Bag 5. Weigh 7 missing? Bag 7. One weighing tells you everything.

> ANSWER
>
> Take k coins from bag k, weigh all 55 together, and read off 550 − (scale reading). That difference is the light bag's number. One weighing, no searching.

WHY IT MATTERS

A neat counting trick: you can pack ten possible answers into a single number.

Problem 2-48 PUZZLE

IN PLAIN WORDS

Eight balls look identical, but one is slightly heavier. Using a balance scale only **twice**, find the heavy one.

A balance scale gives three possible answers each time: left heavier, right heavier, or level. So two weighings can tell apart up to 3 × 3 = 9 things, and 9 ≥ 8. That count tells you it must be possible. Here is how. Split the 8 balls into groups of **3, 3 and 2**.

1. **Weighing 1:** put the two groups of 3 on the pans.
   - If they *balance*, the heavy ball is one of the 2 set aside. Weigh those two against each other — the heavier pan shows it. Done in two.
   - If one side is *heavier*, the heavy ball is among those 3.
2. **Weighing 2** (heavy ball is in a group of 3): take that group and weigh 1 ball against another 1. If one is heavier, that is it; if they balance, it is the third ball you left off.

> ANSWER
>
> Split the balls 3 / 3 / 2 and you always finish in two weighings. It works because each weighing has 3 outcomes, so two weighings carry $3^2 = 9$ results — more than the 8 balls to tell apart.

WHY IT MATTERS

It shows how counting outcomes ahead of time tells you a puzzle is solvable before you find the exact moves.

Problem 2-49 INTERVIEW

IN PLAIN WORDS

n companies keep merging, two at a time, until only one is left. How many merges does that take, and how many different orders can the merges happen in?

The easy half first: each merge turns two companies into one, so it shrinks the count by exactly one. Going from n down to 1 always takes **n − 1 merges**, whatever the order.

The harder half is: how many different *orders* can the merges play out in? Do the small cases by hand:

N	WAYS	WHY
2	1	only one pair to merge
3	3	first merge is one of the 3 pairs (AB, AC, BC); then only one pair is left
4	18	more choices at each of the 3 merges

The formula that produces 1, 3, 18, … is:

$$\text{number of merge orders} = n! \cdot (n-1)! \,/\, 2^{n-1}$$

Check it: n = 2 gives 2 · 1 / 2 = 1; n = 3 gives 6 · 2 / 4 = 3; n = 4 gives 24 · 6 / 8 = 18. All match.

> ANSWER
>
> Always n − 1 merges, and the number of distinct merge orders is $n!(n-1)! \,/\, 2^{n-1}$, matching 1, 3, 18 for n = 2, 3, 4.

WHY IT MATTERS

It shows that the same number of steps can happen in a huge number of different orders — counting them is its own skill.

Problem 2-50 INTERVIEW

IN PLAIN WORDS

A **Ramanujan number** is one that can be written as the sum of two cubes in *two different ways*: $a^3 + b^3 = c^3 + d^3$. Find all of them with a, b, c, d below n — faster than trying every four numbers.

The lazy way is four nested loops over a, b, c, d — that is $O(n^4)$, hopeless. The trick is the same "remember what you have seen" idea from the fast anagram and two-sum solutions: **build every two-cube sum once and look for collisions.**

Go over every pair (a, b) with $a \le b < n$ (about $n^2/2$ of them), compute $a^3 + b^3$, and drop it into a dictionary whose *key* is that sum. Any key that ends up with two or more different pairs is a Ramanujan number.

```
def ramanujan(n):
    seen = {}                    # sum -> list of (a, b) pairs
    for a in range(1, n):
        for b in range(a, n):    # a <= b avoids counting a pair twice
            seen.setdefault(a**3 + b**3, []).append((a, b))
    return {s: p for s, p in seen.items() if len(p) >= 2}

print(sorted(ramanujan(20)))
```

[(1729, [(1, 12), (9, 10)])]

The smallest is the famous **$1729 = 1^3 + 12^3 = 9^3 + 10^3$** — the "taxicab number" from the Hardy–Ramanujan story. Building the dictionary is one pass over the $\approx n^2/2$ pairs, each insert $O(1)$ on average, so the whole thing is **$O(n^2)$** time and $O(n^2)$ space.

> ANSWER
>
> Store every $a^3 + b^3$ (for $a \le b < n$) in a dictionary keyed by the sum; the keys hit two or more times are the Ramanujan numbers. Time **$O(n^2)$**, far better than brute-force $O(n^4)$. The smallest is 1729.

WHY IT MATTERS

"Compute a value once and hash it, then look for repeats" turns an $O(n^4)$ search into $O(n^2)$ — the same move behind the fast anagram and two-sum solutions.

Problems 2-51 & 2-52 PUZZLE

IN PLAIN WORDS

Pirates split treasure by rank. The top pirate proposes a split; everyone (including him) votes; if at least half agree it passes, otherwise he is thrown overboard and the next pirate proposes. Pirates are perfectly logical and care most about staying alive, then about money. With $300 and 6 pirates, who gets what (2-51)? And what if there is only **one** indivisible dollar (2-52)?

The trick for every puzzle like this is to **reason backwards** from the smallest case, because when a pirate votes he compares "what I get now" against "what I would get if this proposer is thrown overboard". Call the pirates P1 (most junior) up to P6 (most senior); the senior proposes first. "At least half" means a tie is enough to pass.

Part 2-51, splitting $300. Build it up one pirate at a time:

PIRATES LEFT	VOTES NEEDED	PROPOSER KEEPS	WHO GETS $1 (TO BUY THE VOTE)
2 (P2, P1)	1	$300	nobody — P2's own vote is half of 2
3	2	$299	P1 (who would get $0 if it fell to 2 pirates)
4	2	$299	P2 (who gets $0 in the 3-pirate outcome)
5	3	$298	P3 and P1 (the two who get $0 with 4 pirates)
6	3	$298	P4 and P2 (the two who get $0 with 5 pirates)

Each proposer only bribes the *cheapest* pirates — the ones who would get nothing in the next round — and only just enough of them to reach half the votes. So with 6 pirates, the senior P6 needs 3 votes (his own plus two), and he gets them by handing $1 each to P4 and P2, keeping $298.

Part 2-52, a single indivisible dollar. Now the proposer has almost nothing to bribe with, so *survival* votes do the work. Reason down the chain: with 5 pirates the proposer P5 would need 3 votes but can only bribe one pirate with the single dollar — so a 5-pirate proposal fails and P5 would be thrown overboard. That fact is the key: P5 is desperate to never reach a 5-pirate game.

So when 6 pirates face the single dollar, the senior P6 needs 3 votes and gets them like this: his own vote; P5's vote for free (because if P6 goes overboard the game drops to 5 pirates where P5 dies, so P5 backs almost anything that keeps P6 afloat); and one

more vote bought by giving the single dollar to one pirate who would otherwise get nothing (P4, P3, or P1). That is three votes — it passes.

One honest caveat (the assumption behind 2-52). The single-dollar answer needs a tie-breaking rule. Skiena states only “survival first, then money”, which leaves open what a pirate does when a proposal gives him *the same* outcome (same survival, same money) as letting the proposer drown. The clean “nobody dies” result assumes an indifferent pirate votes **for** the proposal (equivalently, would rather not throw a colleague overboard for nothing). Under the opposite tie-break the split shifts and some proposers do die. So the unique backward-induction answer requires stating that extra preference.

> ANSWER
>
> (2-51) The top pirate keeps $298 and gives $1 each to P4 and P2, passing 3-to-3; P5, P3, P1 get nothing. (2-52) The top pirate survives and no one need die: he gives the one dollar to a single pirate whose vote he needs, and relies on P5's free vote (P5 votes yes just to avoid dying next round).

WHY IT MATTERS

It shows the power of working backwards from the simplest case — the same habit behind many algorithms.

Programming Challenges

Programming Challenges PRACTICE

IN PLAIN WORDS

Three short online-judge (UVA) problems that go with this chapter. Each needs one clever observation, not heavy machinery. Here is a plain approach for each.

Primary Arithmetic — count the carries when you add two numbers. Add them the grade-school way, digit by digit from the right, keeping a running carry. Every column where the two digits plus the incoming carry reach 10 makes a carry into the next column. Count those events. It is O(number of digits) — really just "do primary-school addition and watch the carries", then report none, one, or the count.

A Multiplication Game — two players start from a product of 1 and take turns multiplying it by any number from 2 to 9; the first to reach or pass `n` wins, and Stan goes first. Solve it by *reasoning backwards*. With best play the winner depends only on which "band" `n` falls in, and the band edges are **9, 18, 162, 324, 2916, …** — the gaps alternate **×2 then ×9** (not "powers of nine"): Stan wins for `n` in `2–9`, the second player for `10–18`, Stan for `19–162`, the second player for `163–324`, Stan for `325–2916`, and so on. A clean way to decide: repeatedly replace `n` by `ceil(n / 18)` until `n ≤ 18`; then **Stan (first) wins exactly when the leftover `n ≤ 9`**. (The special case `n = 1` is a second-player win.) No search needed — a handful of divisions.

Light, More Light — lamps 1..N start off; person i flips every lamp whose number is a multiple of i, so lamp k is flipped once for each of its divisors. A lamp ends *on* only if it was flipped an odd number of times, i.e. if k has an odd number of divisors. Divisors normally come in pairs (d and k/d), *except* when k is a perfect square (the square root pairs with itself). So the only lamps left on are the perfect squares. The whole problem becomes: is k a perfect square? Check with an integer square root, O(1) per query.

> ANSWER
>
> Primary Arithmetic: count carries during digit-by-digit addition. A Multiplication Game: the winner is decided by which band `n` falls in, where the band edges 9, 18, 162, 324, 2916… grow by alternating ×2 and ×9 (decide by dividing `n` by 18 repeatedly, then testing `n ≤ 9`). Light, More Light: the lamps left on are exactly the perfect squares.

WHY IT MATTERS

Each shows that the right observation turns a scary-looking problem into a few lines of code.

Companion to Algorithm Analysis with Python. Problems from Steven Skiena, *The Algorithm Design Manual* (Chapter 2). Explanations written for this course at a beginner level; any errors are the course's, not the book's.